

Алгоритмы и структуры данных

2 семестр (ITMO IS)

Обход в глубину (DFS)

DFS строит дерево обхода, заходя в каждого соседа до упора, а затем откатываясь назад.

```
void dfs(int v) {
    used[v] = true;
    for(int to: g[v])
        if(!used[to]) dfs(to);
}
```

Время: $O(V + E)$.

Обход в ширину (BFS)

BFS обходит граф слоями от стартовой вершины и поэтому в невзвешенном графе сразу даёт кратчайшее расстояние в числе рёбер.

```
queue<int> q;
q.push(s); dist[s]=0;
while (!q.empty()){
    int v = q.front();
    q.pop();
    for(int to: g[v])
        if(dist[to]==INF {
            dist[to]=dist[v] + 1;
            p[to] = v;
            q.push(to);
        }
}
```

Время: $O(V + E)$.

Топологическая сортировка

Топологическая сортировка задаёт порядок вершин в DAG, при котором каждое ориентированное ребро идёт из более ранней вершины в более позднюю.

```
vector<int> order(v);
vector<int> used(v);
void dfs(int v){
    used[v] = 1;
    for(int to: g[v]) {
        if(!used[to]) dfs(to);
    }
    order.push_back(v);
}

void dag() {
    for (int i = 0; i < v; i++) {
        if(!used[i]) {
            dfs(i);
        }
    }
    reverse(order.begin(), order.end());
}
```

Время: $O(V + E)$.

Конденсация графа. Поиск компоненты слабой связности [To do]

Сильные компоненты связности объединяют вершины, достижимые друг из друга, а после сжатия каждой такой компоненты получается DAG конденсации. Для слабой связности в орграфе достаточно забыть направления рёбер и запустить обычный обход по неориентированной версии графа.

```
// Kosaraju
run_dfs_on_g(); order by exit time;
run_dfs_on_gr(reverse order) => comp[v];
// weak components: traverse edges as undirected
```

Время: $O(V + E)$.

Поиск и восстановление циклов [To do]

Циклы в орграфе обычно ищут через цвета вершин в DFS: ребро в серую вершину означает обратный заход и наличие цикла. После обнаружения цикл восстанавливают по массиву предков, а в неориентированном графе отдельно учитывают, что ребро к родителю не считается циклом.

```
bool dfs(int v){
    color[v]=1;
    for(int to: g[v]){
        if(color[to]==0){ p[to]=v; if(dfs(to)) return true; }
        else if(color[to]==1){ cyc_start=to; cyc_end=v; return true; }
    }
    color[v]=2; return false;
}
```

Время: $O(V + E)$.

Гамильтонов цикл при достаточных условиях [To do]

Задача о гамильтоновом цикле в общем случае сложная, поэтому на практике часто проверяют достаточные условия существования цикла. Теоремы Дирака и Оре позволяют быстро установить, что цикл точно существует, если степени вершин достаточно велики относительно числа вершин графа.

```
bool dirac=true;
for(v) if(deg[v] < n/2) dirac=false;
if(dirac) cout << "Hamiltonian cycle exists";
```

Время проверки условий:
 $O(n^2)$.

Эйлеров цикл [To do]

Эйлеров цикл проходит по каждому ребру ровно один раз и возвращается в стартовую вершину. В неориентированном графе для его существования нужны чётные степени всех вершин ненулевой степени и связность по этим вершинам, а само построение обычно делают алгоритмом Хиерхольцера.

```
stack<int> st; vector<int> ans;
st.push(start);
while(!st.empty()){
    int v=st.top();
    if(has_edge(v)) {
        int to=take_edge(v);
        st.push(to);
    }
    else {
        ans.push_back(v);
        st.pop();
    }
}
```

Время: $O(E)$.

Компоненты связности

В неориентированном графе

Каждый запуск DFS или BFS из ещё не посещённой вершины выделяет ровно одну компоненту связности. Так можно не только посчитать число компонент, но и пронумеровать вершины по компонентам или собрать вершины каждой компоненты отдельно.

```
for(int v=0; v<n; ++v)
    if(!used[v]){ ++cc; dfs(v); }
```

Время: $O(V + E)$.

В ориентированном графе (Алгоритм Косораю)

1. Первый DFS (по исходному графу) + кладем вершину после обработки
2. Разворачиваем граф
3. Второй DFS (по обратному графу)

Время: $O(V + E)$.

Алгоритм Краскала

Алгоритм строит минимальное остовное дерево жадно: рассматривает рёбра в порядке возрастания веса и добавляет ребро, если оно соединяет разные компоненты. Для быстрой проверки используют DSU, поэтому алгоритм особенно удобен, когда список рёбер уже дан явно.

```
sort(edges.begin(), edges.end());
for(auto [w,u,v]: edges)
    if(find(u)!=find(v)) {
        unite(u,v);
        mst+=w;
    }
```

Время: $O(E \log E)$.

Алгоритм Прима

Прим тоже строит минимальное остовное дерево, но растит его из одной стартовой вершины, каждый раз добавляя минимальное ребро на границе между уже выбранными и ещё не выбранными вершинами. На разреженных графах обычно используют кучу, а на плотных может быть удобен вариант с матрицей.

```
pq.push({0,s});
while(!pq.empty()){
    auto [w,v] = pop_min();
    if(in[v]) continue;
    in[v]=true;
    mst+=w;
    for(auto [to,c]: g[v])
        if(!in[to])
            pq.push({c,to});
}
```

Время: $O(E \log V)$ с кучей

Алгоритм Беллмана—Форда

Беллман—Форд находит кратчайшие пути из одной вершины даже при наличии отрицательных рёбер. Он последовательно расслабляет все рёбра, а дополнительная итерация после $n - 1$ проходов позволяет определить наличие достижимого отрицательного цикла.

```
d.assign(n, INF); d[s]=0;
for(int i=0;i<n-1;i++)
```

Время: $O(V * E)$.

```
for(auto [u,v,w]: edges)
    d[v]=min(d[v], d[u]+w);
```

Алгоритм DAG (кратчайшие пути в DAG)

В DAG кратчайшие пути считаются быстрее, чем в общем случае, потому что после топологической сортировки рёбра можно релаксировать один раз в правильном порядке. Это даёт линейное по размеру графа решение и для положительных, и для отрицательных весов, если цикл отсутствует.

```
topo_sort(); d[s]=0;
for(int v: topo)
    for(auto [to,w]: g[v])
        d[to]=min(d[to], d[v]+w);
```

Время: $O(V + E)$.

Дейкстра (очередь / массив)

Дейкстра ищет кратчайшие пути из одной вершины в графе с неотрицательными весами, каждый раз фиксируя вершину с минимальной текущей дистанцией. Реализация с приоритетной очередью подходит для разреженных графов, а вариант с простым выбором минимума иногда используют на плотных.

```
d.assign(n, INF); d[s]=0;
priority_queue<P, vector<P>, greater<P>> pq;
pq.push({0,s});
while(!pq.empty()){
    auto [dist,v]=pq.top(); pq.pop();
    if(dist!=d[v]) continue;
    for(auto [to,w]: g[v]) if(d[to]>d[v]+w){
        d[to]=d[v]+w; p[to]=v; pq.push({d[to],to});
    }
}
```

Время: $O((V + E) \log V)$
с priority_queue, $O(V^2)$
с массивом.

Флойд—Уоршалл

Флойд—Уоршалл решает задачу кратчайших путей для всех пар вершин с помощью динамики по множеству разрешённых промежуточных вершин. Алгоритм прост в реализации и удобен, когда граф небольшой и нужно знать расстояния между всеми парами сразу.

```
for(int k=0;k<n;k++)
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            d[i][j]=min(d[i][j], d[i][k]+d[k][j]);
```

Время: $O(V^3)$.

Диаметр дерева

Диаметр дерева можно найти двумя обходами: сначала из любой вершины ищут самую далёкую вершину A, а затем из A ищут самую далёкую вершину B. Путь между A и B и есть один из диаметров дерева, а его длина равна максимальному расстоянию.

```
int A = farthest(0);
auto [B, dist] = farthest(A);
```

Время: $O(V)$.

Кун: максимальное паросочетание в двудольном

Алгоритм Куна строит максимальное паросочетание в двудольном графе, пытаясь для каждой левой вершины найти увеличивающий путь. Если правая вершина уже занята, алгоритм пытается переназначить её текущую пару, чтобы освободить место для нового ребра.

```
bool try_kuhn(int v){
    if(used[v])
        return false;
    used[v]=true;
    for(int to: g[v]) {
        if(mt[to]==-1 || try_kuhn(mt[to])) {
            mt[to]=v;
            return true;
        }
    }
    return false;
}
```

Время: $O(V * E)$ в худшем случае.

Форда—Фалкерсон и Эдмондс—Карп (макс. поток) - to do псевдокод эдмондса карпса

Форд—Фалкерсон увеличивает поток, пока в остаточной сети существует путь из истока в сток с положительной пропускной способностью. Эдмондс—Карп конкретизирует этот подход, всегда выбирая кратчайший по числу рёбер увеличивающий путь через BFS, что даёт гарантированную полиномиальную оценку.

```
while(bfs(s,t,parent)){
    int add = bottleneck(parent);
    augment(parent, add);
    flow += add;
}
```

Время: для Edmonds—Karp $O(V * E^2)$.

Форда—Фалкерсон для max matching в двудольном

Максимальное паросочетание в двудольном графе можно свести к задаче максимального потока: добавляют исток, сток и рёбра единичной пропускной способности. После этого любой алгоритм maxflow автоматически находит размер паросочетания как величину максимального потока.

```
build_network(); // unit capacities
int matching = maxflow(s,t);
```

Время: как у выбранной реализации maxflow.

Алгоритм масштабирования потока

Масштабирование потока ускоряет поиск максимального потока при целых пропускных способностях: сначала ищут увеличивающие пути только по рёбрам остаточной сети с большой пропускной способностью, а затем постепенно уменьшают порог delta. Это уменьшает число «мелких» увеличений потока и часто работает быстрее обычного Форда—Фалкерсона.

```
int flow = 0;
int delta = highest_power_of_two_leq(U);
while(delta > 0){
    while(bfs_with_limit(s, t, delta, parent)){
        int add = bottleneck(parent);
        augment(parent, add);
        flow += add;
    }
    delta >>= 1;
}
```

Время: $O(E^2 \log U)$ для целых пропускных способностей, где U — максимальная capacity.

Минимальное вершинное покрытие в двудольном графе

В общем графе задача минимального вершинного покрытия NP-трудна, но в двудольном графе она решается через максимальное паросочетание по теореме Кёнига. После нахождения максимального паросочетания запускают обход по чередующимся путям из непокрытых вершин левой доли, а затем берут покрытие $(L \setminus Z_L) \cup (R \cap Z_R)$.

```
run_kuhn();
dfs_alt_from_unmatched_left();
cover = (L - visL) union (R & visR);
```

Построение покрытия после matching: $O(V + E)$. Итого: как поиск max matching, например $O(V * E)$ с Куном.

Максимальное независимое множество вершин в двудольном графе

В двудольном графе максимальное независимое множество является дополнением минимального вершинного покрытия. Поэтому достаточно найти минимальное покрытие, а затем взять все вершины, которые в него не вошли; по размеру получается $|MIS| = |V| - |MVC| = |V| - |MM|$.

```
cover = min_vertex_cover_bipartite();
for(int v = 0; v < n; ++v)
    if(!in_cover[v])
        indep.push_back(v);
```

Время: дополнение строится за $O(V)$. Итого: как поиск min vertex cover / max matching.

Мосты и точки сочленения

Мосты и точки сочленения ищут через один DFS, поддерживая время входа tin и значение low, которое показывает, насколько высоко по дереву можно подняться обратными рёбрами. Если из поддерева нельзя вернуться выше текущей вершины, соответствующее ребро или вершина оказываются критическими для связности.

```
void dfs(v,p=-1){
    used[v]=1;
    tin[v]=low[v]=timer++;
    for(int to: g[v]) {
        if(to != p) {
            if(used[to]) {
                low[v]=min(low[v], tin[to]);
            }
            else {
                dfs(to,v);
                low[v]=min(low[v], low[to]);
                if(low[to] > tin[v])
                    bridge(v,to);
            }
        }
    }
}
```

Время: $O(V + E)$.

Жадная раскраска графа

Жадная раскраска проходит вершины в некотором порядке и каждой вершине назначает минимальный допустимый цвет, который не конфликтует с уже раскрашенными соседями.

```
vector<int> color(n, -1);
for (int v = 0; v < n; ++v) {
    vector<bool> used(n, false);
```

Время: $O(V + E)$

```
for (int to : g[v]) {  
    if (color[to] != -1) {  
        used[color[to]] = true;  
    }  
}  
for (int c = 0; c < n; ++c) {  
    if (!used[c]) {  
        color[v] = c;  
        break;  
    }  
}  
}
```